

Creative Programming: Interaction

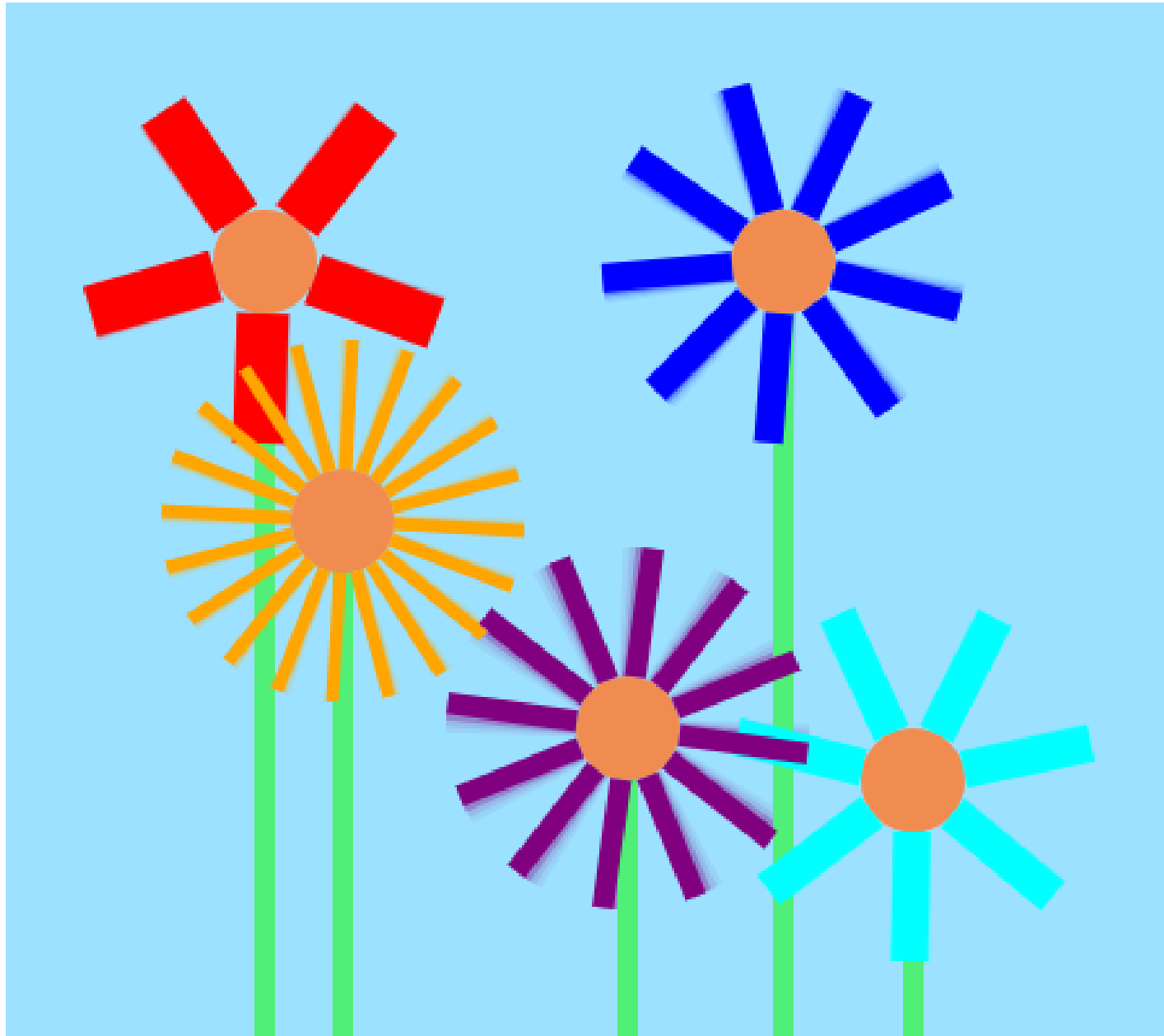
Lecture 4 (17 Feb)

Narration

Table of contents

- Angle recap
- The World of Functions
- Mouse
- Keyboard

Narration



Narration

(Re)visiting slides?

You can jump to any slide with the menu to the (bottom) left.

Did you miss a slide or want to revisit?

Open the narration tab while studying to get an explanation of difficult slides.



Course improvements

- Thank you for your feedback!
- We have had the fundamentals so we will have more time on practicing in class
- There is now a coding reference on GitHub with details on what we have covered so far

Narration

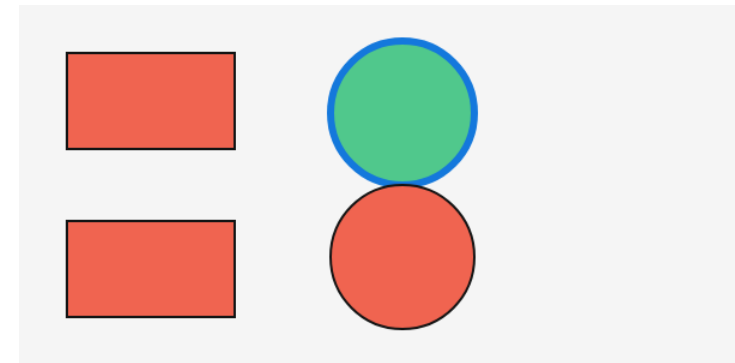
Angle recap

Narration

Push & Pop

- Properties like `fill()` and `stroke()` affects all subsequent shapes
- `push()` and `pop()` allow us to save and restore the current drawing state
- Any changes you make between `push()` and `pop()` will not affect the drawing state outside of that block

```
1 ▾ function setup() {  
2   createCanvas(600, 300);  
3 }  
4  
5 ▾ function draw() {  
6   background(245);  
7  
8   // Global drawing state.  
9   stroke(20);  
10  strokeWeight(2);  
11  fill(240, 100, 80);
```

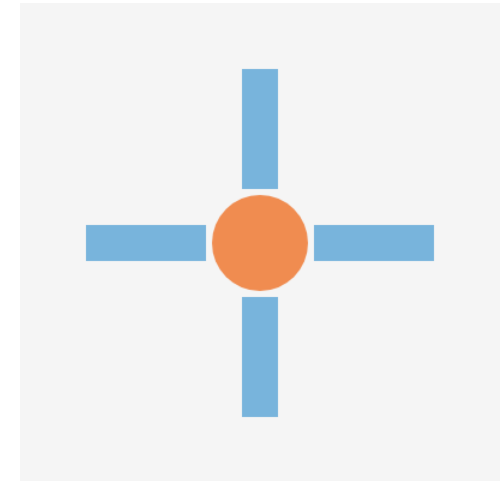


Narration

Straight angles

- If we just use `rect()`, we can only get rectangles on straight angles
- Without angled rectangles our flower sketch would look like this:

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3   rectMode(CENTER); // Draw rectangles from center  
4   noStroke();  
5 }  
6  
7 ▾ function draw() {  
8   background(245);  
9   let cx = width / 2;  
10  let cy = height / 2;  
11  
12   // Center of flower  
13   fill(240, 140, 80);  
14   ellipse(cx, cy, 80, 80);  
15  
16   // Details
```

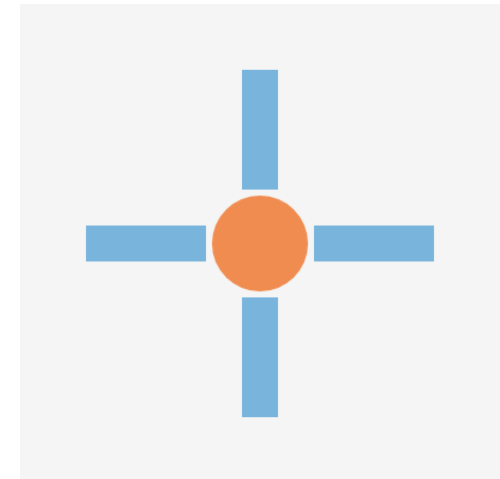


Narration

Translate

- In order to rotate we need to set the origin using `translate()`
- `translate(x, y)` moves the origin to the specified (x, y) position
- For a flower we would for example want to move the origin to the center

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3   rectMode(CENTER); // Draw rectangles from center  
4   noStroke();  
5 }  
6  
7 ▾ function draw() {  
8   background(245);  
9  
10  // Move origin to center of flower  
11  translate(width / 2, height / 2);  
12  
13  // Center of flower
```

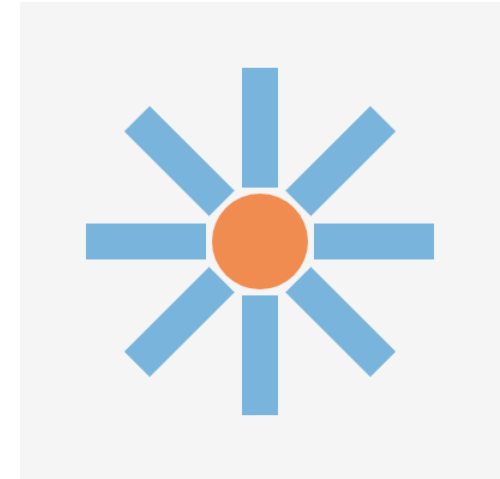


Narration

Rotate

- With our new origin we can now rotate our shapes using `rotate()`
- `rotate(angle)` rotates the coordinate system by the specified angle
- Make sure to use `angleMode(DEGREES)` first if you want to use degrees

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3   angleMode(DEGREES); // makes it easier to think in 0-360 degrees  
4   rectMode(CENTER); // Draw rectangles from center  
5   noStroke();  
6 }  
7  
8 ▾ function draw() {  
9   background(245);  
10  
11   // Move origin to center of flower  
12   translate(width / 2, height / 2);  
13
```

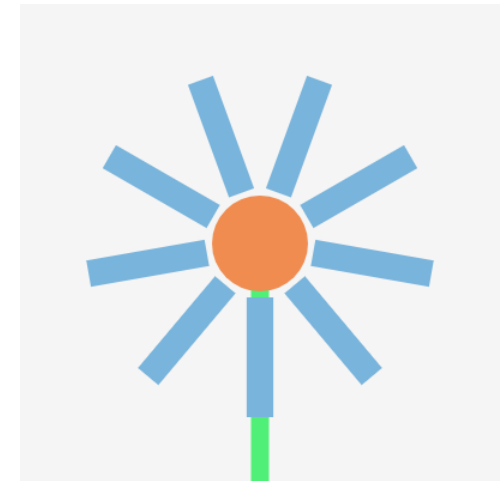


Narration

For-loop

- We can automate the drawing of multiple petals using a for-loop
- By rotating the coordinate system by a fixed angle in each iteration, we can create a circular pattern of petals

```
1
2 let petal_count = 9; // change this to get more or fewer petals
3
4 ▾ function setup() {
5   createCanvas(400, 400);
6   angleMode(DEGREES); // makes it easier to think in 0-360 degrees
7   rectMode(CENTER); // Draw rectangles from center
8   noStroke();
9 }
10
11 ▾ function draw() {
12   background(245);
13   translate(width / 2, height / 2); // move origin to center of flower
14
```



Narration

Flower!

💡 Class Assignment: Flower!

1. Create a flower sketch using `translate()`, `rotate()`, and a for-loop to draw multiple petals
2. Can you make it so that you can quickly change the location (x,y) and number of petals?
3. Can you make the petals spin using `frameCount`?



Narration

The World of Functions

Narration

Shape functions

- `rect()` and `ellipse()` are examples of functions that draw shapes on the canvas
- What if we want to repeatedly draw a custom shape?

Narration

Vertices

- We can define custom shapes with vertices
- Start and end your shape with `beginShape()` and `endShape(CLOSE)`
- Define every vertex with `vertex(x, y)`

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3 }  
4  
5 ▾ function draw() {  
6   background(245);  
7  
8   // Set origin to center  
9   translate(width / 2, height / 2);  
10  
11   fill(240, 200, 80);  
12   stroke(20);  
13   strokeWeight(2);
```



Narration

Extra: Round vertices

- We can also round the corners of our shapes via `bezierVertex()`
- This function takes two control points and an anchor point, which defines a curve between the previous vertex and the anchor point
- `bezierVertex(x1, y1, x2, y2, x3, y3)` where (x1, y1) and (x2, y2) are the control points and (x3, y3) is the final destination anchor point

```
1 ▾ function setup() {  
2   createCanvas(500, 500);  
3 }  
4  
5 ▾ function draw() {  
6   background(245);  
7   fill("red");  
8   stroke(20);  
9   strokeWeight(2);
```



Narration

Repeatedly drawing shapes

- Vertex definitions take up a lot of space
- If we want to draw multiple shapes, we clutter the code quickly
- Solution: functions

Narration

Functions

- Functions are reusable blocks of code that perform a specific task
- They allow us to organize our code and avoid repetition
- `rect()` and `ellipse()` are examples of built-in functions, but we can also create our own custom functions!

Narration

Function syntax

- Functions have a name and take a list of parameters(inputs):

```
1 function functionName(a, b, c){  
2     // code to be executed  
3 }
```

- In this above example, (a, b, c) are the parameters of the function. These are essentially variables for within the function's scope

Narration

Function calling

- We can now call the function like this:

```
1 functionName(1, 2, 3);
```

- In this case, we are passing the values 1, 2, and 3 as arguments to the function, which will be assigned to the parameters a, b, and c respectively when the function is executed

Narration

Function return

- Functions can also return a value using the `return` keyword
- This allows us to use the output of a function in other parts of our code

```
1 function add(a, b){  
2   return a + b;  
3 }
```

- This example adds two numbers and returns the result. We can use this function like this:

```
1 let sum = add(5, 10);  
2 console.log(sum); // Output: 15
```

Narration

Custom function

- Going back to our vertex example, we can move this code to a function
- If we use parameters, we can reuse the function to draw multiple shapes on different locations

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3 }  
4  
5 ▾ function star(x, y) {  
6   // Move to the location without affecting the global state  
7   push();  
8   translate(x, y);  
9  
10  beginShape(); // start defining a custom shape  
11  vertex(0, -40); // top point  
12  vertex(12, -12);  
13  vertex(40, -12); // right outer point  
14  vertex(16, 8);
```

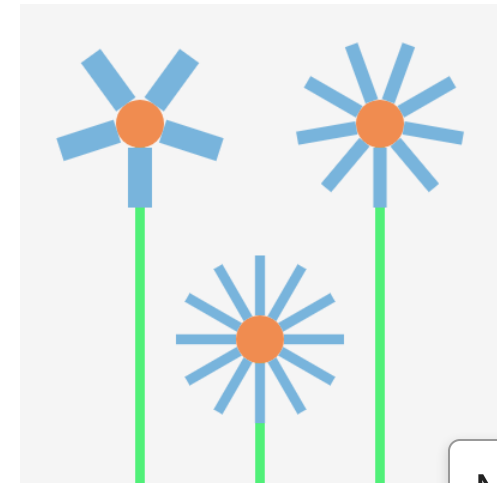


Narration

Flowers and functions

- We can now use our custom function to draw multiple flowers on the canvas
- Move the flower drawing code to a function and use parameters to specify the location and number of petals

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3   angleMode(DEGREES); // makes it easier to think in 0-360 degrees  
4   rectMode(CENTER); // Draw rectangles from center  
5   noStroke();  
6 }  
7  
8 ▾ function flower(x, y, petal_count) {  
9   push();  
10  translate(x, y); // move origin to center of flower  
11  
12  let petal_angle = 360 / petal_count; // angle between each petal
```



Narration

Flower forest

💡 Class Assignment: Flower forest

1. Move flower code from the previous assignment to a function
2. Make it so that you can specify the location and number of petals as parameters
3. Use this function to draw a flower forest with multiple flowers on the canvas
4. Can you add more parameters? For example, can you specify the size of the flower or the colours?



Narration

Mouse

Narration

What we have seen so far

- We have used `mouseX` and `mouseY` to make our sketches interactive
- We also used `pmouseX` and `pmouseY` to get the previous mouse position
- Can we do more with the mouse?

Narration

Events

- `setup()` is a built-in function that happens when the program starts
- Some built-in functions are called automatically when certain *events* happen
- For example, `mousePressed()` is called whenever the mouse is pressed
- We can define our own `mousePressed()` function to specify what should happen when the mouse is pressed

Narration

Mouse events

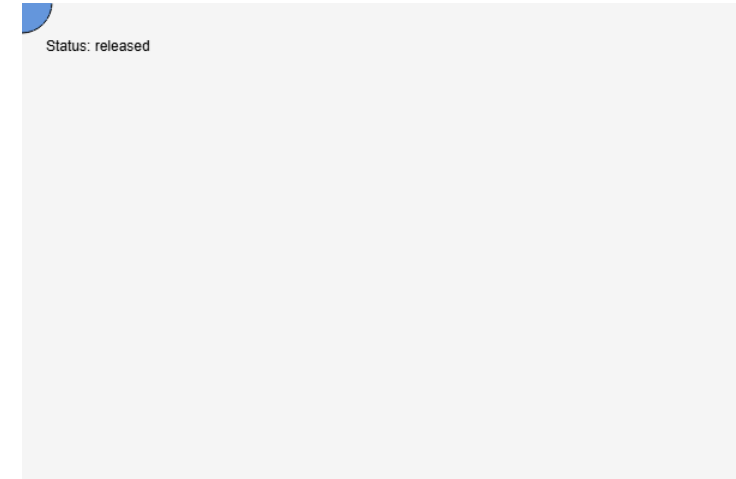
- `mousePressed()`: called when the mouse button is pressed
- `mouseReleased()`: called when the mouse button is released
- `mouseClicked()`: called when the mouse button is clicked
- `mouseMoved()`: called when the mouse is moved
- `mouseDragged()`: called when the mouse is dragged (moved while down)
- `mouseWheel()`: called when the mouse wheel is scrolled.
 - End with `return false;` to prevent page scrolling

Narration

Programming with mouse events

- Let us look at some of these events in action

```
1 let isPressed = false;
2 let circleSize = 50;
3
4 ▾ function setup() {
5   createCanvas(600, 400);
6 }
7
8 ▾ function draw() {
9   background(245);
10
11   // Draw a circle that changes based on mouse state
12   fill(100, 150, 220);
13   ellipse(mouseX, mouseY, circleSize, circleSize);
14
15
16   // Display current state
17   fill(0);
18   textSize(12);
```



Narration

Mouse event parameters

- Some mouse events also have parameters that provide additional information about the event
- For example, the `mouseWheel()` event has an optional `event` parameter that contains information about the scroll amount and direction
 - The speed and direction of the scroll can be accessed with `event.delta`
- Check out the (p5.js) reference for more details on mouse events and their parameters

Narration

Easing

- Easing is a technique used to create smooth animations by gradually changing a value over time
- Instead of instantly changing a value to a new target, we can “ease” towards the new one

```
1 let currentValue = 0;  
2 function draw() {  
3   let targetValue = 100; // The value we want to reach  
4   currentValue = 0.9 * currentValue + 0.1 * targetValue; // Easing towards the target value  
5 }
```

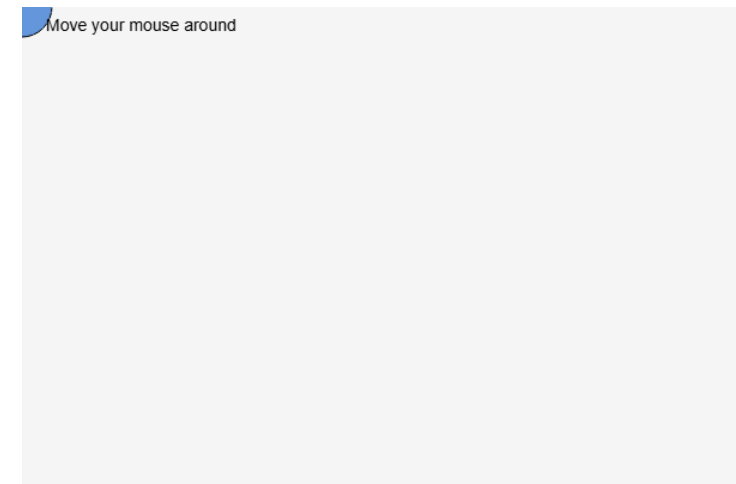
- Here we replace 10% of the location with the target location in each frame, which creates a smooth transition towards the target.

Narration

Easing example

In this example we ease a circle towards the mouse position. The circle will smoothly follow the mouse around the canvas.

```
1  let x = 0;
2  let y = 0;
3
4  function setup() {
5    createCanvas(600, 400);
6    x = width / 2;
7    y = height / 2;
8  }
9
10 function draw() {
11   background(245);
12
13   // Apply easing formula to x
14   // currentValue = 0.9 * currentValue + 0.1 * targetValue
15   x = 0.9 * x + 0.1 * mouseX;
16
17   // Apply easing formula to y
```



Narration

Extra: Mouse state outside of events

- If you want to check the state of the mouse outside of the event functions, you can use the `mouseIsPressed` variable
- This variable is `true` if the mouse is currently being pressed, and `false` otherwise
- `mouseButton` contains the value of the mouse button that is currently being pressed (LEFT, RIGHT, or CENTER)

```
1 function draw() {  
2   if (mouseIsPressed) {  
3     console.log(mouseButton); // Output: LEFT, RIGHT, or CENTER depending on which button is pressed  
4   }  
5 }
```

Narration

Mousing around

💡 Class Assignment: Mousing around

1. Create an interactive sketch, or modify a previous one, with two different mouse events
2. Experiment with the parameters of the mouse events to create interesting interactions
3. Can you make the mouse wheel change the size of things in your sketch?



Creative Programming 2026 Lecture 4 - Ties Robroek - IT University of Copenhagen

Narration

Keyboard

Narration

Keyboard events

- Similar to mouse events, there are also keyboard events that we can use to make our sketches interactive
- `keyPressed()`: called when a key is pressed
- `keyReleased()`: called when a key is released
- `keyTyped()`: called when a key is typed (pressed and released)
- These events update `key` and `keyCode`, two variables that contain information about the key that was pressed
- Check out the (p5.js) reference for more details on keyboard events and their parameters

Narration

Key and keyCode

- `key` contains the value of the key that was pressed as a string. For example, if you press the 'a' key, `key` will be 'a'
- `keyCode` contains the numeric code of the key that was pressed. For example, the 'a' key has a `keyCode` of 65
 - Some keys have variables that hold their keycodes, such as `SPACE`, `ENTER`, and `LEFT_ARROW`.

Narration

Key example

- In the snippet below we check whether the 'a' key, space bar, or left arrow key is pressed and do something in each case

```
1 function keyPressed() {  
2   if (key == 'a') {  
3     // Do something when the 'a' key is pressed  
4   } else if (keyCode == 32) { // We could have also used SPACE instead of 32  
5     // Do something when the space bar (keyCode 32) is pressed  
6   } else if (keyCode == LEFT_ARROW) {  
7     // Do something when the left arrow key is pressed  
8   }  
9 }
```

Narration

A useful snippet

- We prefer using `key` for letter keys and `keyCode` for special keys
- With a small tool, though, we can find the keycode for any key

```
1 ▾ function setup() {  
2   createCanvas(400, 400);  
3   textSize(32);  
4   text("Type something!", 50, 100);  
5 }  
6  
7 ▾ function keyPressed() {  
8   // Run whenever there is a key press event  
9   clear()  
10  text("Type something!", 50, 100);  
11  text('"' + str(key) + '"', 50, 200);  
12  text(str(keyCode), 50, 300);  
13  console.log(key, keyCode);  
14 }  
15
```

Type something!

Narration

Programming with keyboard events

- Let us look at some of these events in action

```
1  let lastCharacter = '';  
2  let backgroundColor = 245;  
3  
4  function setup() {  
5    createCanvas(600, 400);  
6    textSize(24);  
7  }  
8  
9  function draw() {  
10   background(backgroundColor);  
11  
12   // Display instructions  
13   fill(0);  
14   text("Try: Press and release keys, type characters", 20, 20);  
15  
16   // Display last character typed  
17   fill(0);  
18   text("Last character typed: " + lastCharacter, 20, 80);
```

Try: Press and release keys, type characters

Last character typed:

Press S to change background color
Press R to reset

Narration

Extra: Key state outside of events

- If you want to check the state of the keyboard outside of the event functions, you can use the `keyIsDown()` function
- This function takes a key code as an argument and returns `true` if that key is currently being pressed, and `false` otherwise

```
1 function draw() {  
2   if (keyIsDown(LEFT_ARROW)) {  
3     // Do something while the left arrow key is being pressed  
4   }  
5 }
```

Narration

Key controls

- One of the most common uses of keyboard input is to control an object
- In this example, we can move the ball with the arrow keys

```
1  let x = 200;
2  let y = 200;
3  let speed = 5;
4
5  function setup() {
6    createCanvas(600, 400);
7  }
8
9  function draw() {
10   background(245);
11
12   // Check arrow keys and move the circle
13   if (keyIsDown(LEFT_ARROW)) {
14     x -= speed; // move left
15   }
16   if (keyIsDown(RIGHT_ARROW)) {
```

Use arrow keys to move the circle
(may need to click canvas first)



Narration

Alphabet

💡 Class Assignment: Alphabet

1. Create a sketch that responds to keyboard input by displaying the `key` variable on the canvas
2. Make an array with the letters of the alphabet (or another word). Make it so the user has to type the letters in the correct order.
3. Can you make it so that the user has to type the letters in reverse order? Or in a random order?



Narration